

Relatório 1 - Multiplicação de Matrizes

1. Enunciado

Implemente duas versões da multiplicação de matriz por vetor (MxV) em C:

- Uma com acesso à matriz por linhas (linha externa, coluna interna).
- Outra por colunas (coluna externa, linha interna).

Meça o tempo de execução de cada versão com uma função apropriada e execute testes com diferentes tamanhos de matriz. Identifique a partir de que tamanho os tempos passam a divergir significativamente e explique por que isso ocorre, relacionando suas observações com o uso da memória cache e o padrão de acesso à memória.

2. Objetivo da Tarefa

O objetivo da tarefa busca comprovar que a ordem em que os dados são lidos da memória impacta o desempenho do algoritmo, evidenciando o funcionamento da **memória Cache** e o princípio da **localidade espacial**.

3. Metodologia

A solução foi implementada na linguagem C e dividida em duas abordagens:

1. **Acesso por Linhas:** O laço externo itera sobre as linhas e o interno sobre as colunas ($A[i][j]$).
2. **Acesso por Colunas:** O laço externo itera sobre as colunas e o interno sobre as linhas ($A[i][j]$).

Para testar o impacto na memória, a matriz foi alocada dinamicamente no *Heap* (usando `malloc`) para suportar tamanhos grandes sem causar *Stack Overflow*. O tempo de execução de cada função foi medido utilizando a biblioteca `<time.h>` (função `clock()`). Os testes foram realizados variando a dimensão da matriz (N), com tamanhos de 100, 1000 e 2000 elementos.

4. Código-Fonte

```
#include <stdio.h>
#include <stdlib.h>
#include <time.h>

// Função 1: Acesso por Linhas
void multiplica_linhas(int N, double A[N][N], double X[N], double Y[N]) {
    for (int i = 0; i < N; i++) {
        Y[i] = 0.0;
        for (int j = 0; j < N; j++) {
            Y[i] += A[i][j] * X[j];
        }
    }
}

// Função 2: Acesso por Colunas
void multiplica_colunas(int N, double A[N][N], double X[N], double Y[N]) {
    for (int i = 0; i < N; i++) {
        Y[i] = 0.0;
    }

    for (int j = 0; j < N; j++) {
        for (int i = 0; i < N; i++) {
            Y[i] += A[i][j] * X[j];
        }
    }
}
```

```

    }
}

int main() {
    int N = 2000; // Tamanho da matriz variavel para os testes

    // Alocação das matrizes/vetores
    double (*A)[N] = malloc(sizeof(double[N][N]));
    double *X = malloc(N * sizeof(double));
    double *Y = malloc(N * sizeof(double));

    // Preenchimento com valores iniciais
    for (int i = 0; i < N; i++) {
        X[i] = 1.0;
        for (int j = 0; j < N; j++) {
            A[i][j] = 2.0;
        }
    }

    // Teste 1: Linhas
    clock_t inicio = clock();
    multiplica_linhas(N, A, X, Y);
    clock_t fim = clock();
    double tempo_linhas = (double)(fim - inicio) / CLOCKS_PER_SEC;

    // Teste 2: Colunas
    inicio = clock();
    multiplica_colunas(N, A, X, Y);
    fim = clock();
    double tempo_colunas = (double)(fim - inicio) / CLOCKS_PER_SEC;

    // Exibição dos Resultados
    printf("Tamanho da Matrix = %d x %d\n", N, N);
    printf("Tempo por Linhas : %f segundos\n", tempo_linhas);
    printf("Tempo por Colunas: %f segundos\n", tempo_colunas);

    // Liberação de memória
    free(A);
    free(X);
    free(Y);

    return 0;
}

```

5. Resultados e Análises

Ao executar o código com diferentes tamanhos de N, observamos o seguinte comportamento nos tempos de execução (valores aproximados):

Tamanho da Matriz (N)	Tempo por Linhas	Tempo por Colunas
100x100	0.000068 segundos	0.000061 segundos
1000x1000	0.005862 segundos	0.002809 segundos
2000x2000	0.024909 segundos	0.014632 segundos

5.1 Por que os tempos divergem?

A linguagem C armazena matrizes na memória em um formato chamado Row-Major Order (ordem por linha). Isso significa que os elementos de uma mesma linha ficam fisicamente adjacentes (colados) na memória RAM.

No acesso por linhas: Quando o processador solicita o elemento $A[0][0]$, ele não traz apenas um dado da RAM, mas traz um bloco inteiro de dados contíguos (uma Cache Line) para a memória Cache L1, que é extremamente rápida. Quando o laço avança para $A[0][1]$, o dado já está na Cache, gerando um Cache Hit (acerto). Isso resulta em máxima velocidade.

No acesso por colunas: Ao pedir o $A[0][0]$, o sistema traz a primeira linha inteira para a Cache. Porém, a próxima iteração solicita o $A[1][0]$ (elemento da segunda linha). Esse dado está fisicamente distante na memória RAM e não está na Cache. O processador sofre um Cache Miss (falta) constante, sendo obrigado a descartar a Cache e buscar novos blocos na RAM (que é milhares de vezes mais lenta) a cada iteração.

5.2 A partir de que tamanho ocorre a divergência?

A divergência se torna maior ($N=1000$ e $N=2000$) no exato momento em que o tamanho total da matriz ultrapassa a capacidade de armazenamento da memória Cache do processador. Para $N=100$, a matriz inteira cabe dentro da Cache, portanto, mesmo acessando pela coluna de forma ineficiente, todos os dados já estão no processador, e a diferença de tempo não é notada.

5.3. Escolha das Dimensões da Matriz ($N = 100, 1000, 2000$)

A escolha dos tamanhos de matriz não foi arbitrária. Os valores de N foram definidos para avaliar a eficiência do algoritmo em três cenários distintos de ocupação da hierarquia de memória do processador:

1. $N = 100$ ($100 \times 100 \times 8 \text{ bytes} \approx 80 \text{ KB}$):
 - **Cenário de Borda Inferior (Cache Ideal):** Representa um volume de dados pequeno o suficiente para ser armazenado inteiramente nas memórias Cache rápidas do processador (L1/L2). Os resultados comprovam que, enquanto os dados cabem na Cache, a ordem de acesso (linha ou coluna) não gera impacto expressivo no tempo.
2. $N = 1000$ ($1000 \times 1000 \times 8 \text{ bytes} = 8 \text{ MB}$):
 - **Ponto de Inflexão (Início do Gargalo):** O tamanho da matriz ultrapassa a capacidade das Caches L1/L2 e passa a ocupar a Cache L3/RAM. Neste ponto, o fenômeno de *Cache Miss* na versão por colunas começa a se manifestar com clareza, gerando as primeiras diferenças significativas de desempenho.
3. $N = 2000$ ($2000 \times 2000 \times 8 \text{ bytes} = 32 \text{ MB}$):
 - **Cenário de Estresse (Gargalo da Memória RAM):** O volume de dados excede substancialmente a capacidade da memória Cache. A versão por colunas é forçada a realizar buscas constantes (*Cache Misses*) diretamente na memória RAM (que é ordens de grandeza mais lenta), fazendo o tempo de execução disparar em comparação ao acesso por linhas.

6. Conclusão

A atividade demonstra que a complexidade algorítmica (ambos os códigos são $O(N^2)$) não é o único fator determinante no desempenho de um software. O aproveitamento da hierarquia de memória do hardware, garantindo a Localidade Espacial dos dados para maximizar o uso da memória Cache, é essencial para o desenvolvimento de aplicações de alto desempenho.